

UFC

engine lab

Aula 13 - gRPC

Francisco Anderson de Almada Gomes

Sistemas Distribuídos

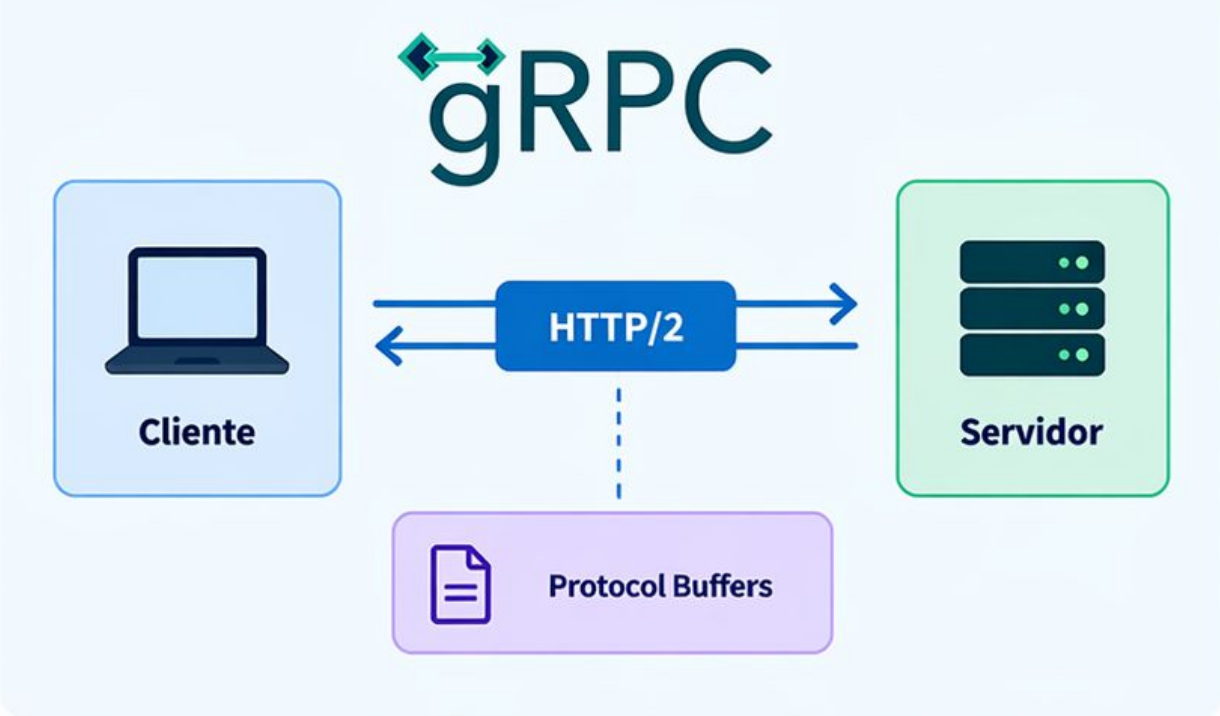
gRPC

RPC
Chamada de métodos entre processos.

Baseado no Protocolo HTTP/2
Multiplexação e comunicação bidirecional.

Protocol Buffers
Serialização de dados em formato binário.

Multi-plataforma
Suporte para diversas linguagens.



UFC

Vantagens

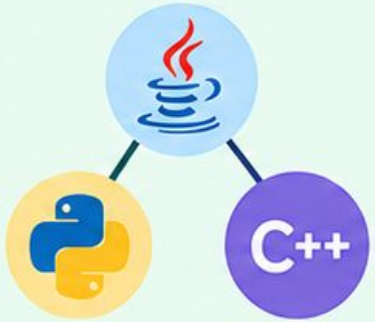
1



Alto desempenho

Mensagens em formato binário e menor overhead.

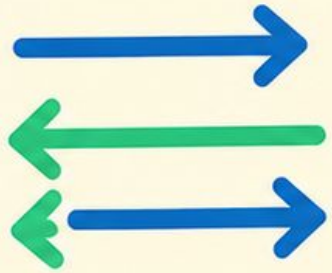
2



Multi-plataforma

Suporte a diversas linguagens.

3



Streaming

Suporte a unary, server, client e bidirecional.

4



Geração de código

Criação automática de clientes e servidores a partir do .proto.

UFC

Protocol Buffer

Definição de mensagens
Estrutura os dados em um arquivo .proto.

Formato binário
Mensagens compactas e eficientes.

Geração de código
Cria classes para diferentes linguagens.

produto.proto

```
syntax = "proto3";

message Produto {
  int32 id = 1;
  string nome = 2;
  double preco = 3;
}
```

Compilação

Mensagem (binário)

```
0A 06 43 61 6D 09 73 61
12 03 31 39 99
1D 00 00 20 41
...
```

UFC

Estrutura de Mensagens Protobuf

produto.proto

```
syntax = "proto3";

message Produto {
  int32 id = 1;
  string nome = 2;
  double preco = 3;
  bool disponivel = 4;
}
```

message
Define uma estrutura de dados.

Campos
Cada campo possui um tipo e um número único.

Numeração
Identifica o campo na serialização.

Tipos de dados
Tipos básicos: int32, string, double, bool, ...

UFC

Exemplo de arquivo .proto

produto.proto

```
syntax = "proto3";

message Produto {
   int32 id = 1;
   string nome = 2;
   double preco = 3;
}

message GetProdutoRequest {
   int32 id = 1;
}

message GetProdutoResponse {
   Produto produto = 1;
}

service ProdutoService {
   rpc GetProduto(GetProdutoRequest) returns (GetProdutoResponse);
}
```

Versão da sintaxe
Utiliza a sintaxe do Protocol Buffers 3.

Mensagem
Define a estrutura de dados e seus campos com tipo e número único.

Mensagens de requisição e resposta
Definem os dados usados no serviço.

Serviço
Define os métodos RPC e os tipos de requisição e resposta.

HTTP/2

Multiplexação
Múltiplas requisições na mesma conexão TCP.

Streams
Requisições e respostas em paralelo.

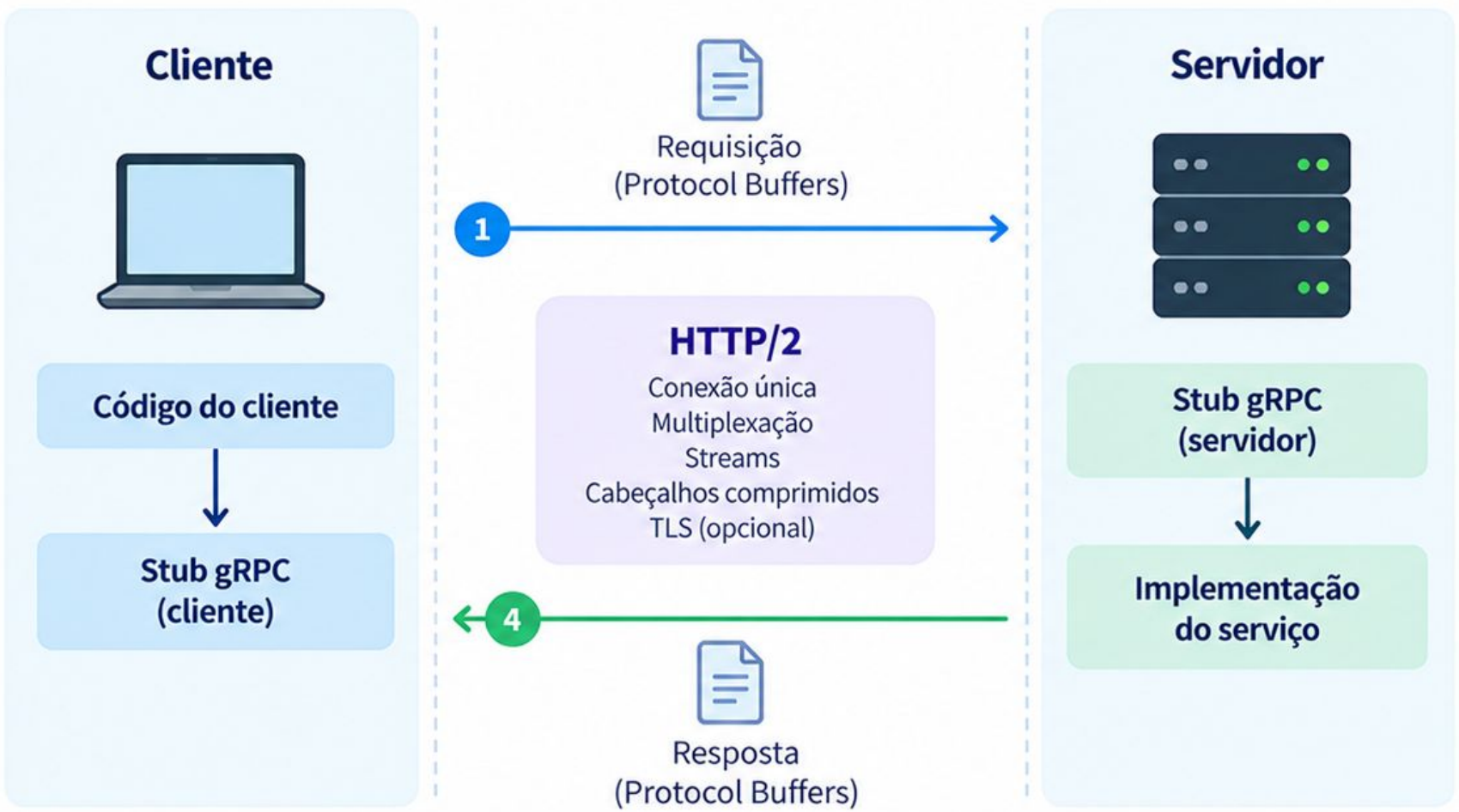
Cabeçalhos comprimidos
Menor uso de banda (HPACK).

Comunicação segura
Utiliza TLS (geralmente na porta 443).



UFC

Funcionamento de uma chamada gRPC



1 Serialização
O cliente chama o método do stub, que serializa a requisição em Protocol Buffers.

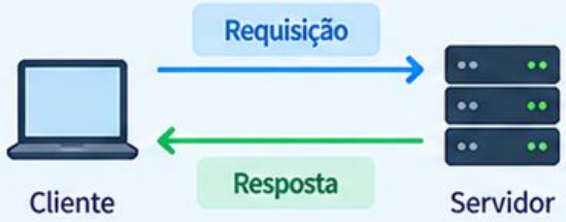
2 Transporte
A requisição é enviada via HTTP/2 em um stream.

3 Processamento
O servidor desserializa a requisição, executa o método e serializa a resposta.

4 Resposta
A resposta é enviada de volta pelo mesmo stream, em formato binário (Protocol Buffers).

Tipos de comunicação gRPC

1 Unary (unária)



O cliente envia uma requisição e recebe uma única resposta.

2 Server streaming



O cliente envia uma única requisição e o servidor envia várias respostas.

3 Client streaming



O cliente envia várias requisições e o servidor retorna uma única resposta.

4 Bidirectional streaming



Cliente e servidor trocam várias mensagens de forma independente e simultânea.

UFC

gRPC vs REST

gRPC Chamadas remotas usando Protocol Buffers sobre HTTP/2



Formato de dados
Protocol Buffers (binário e compacto)

Protocolo
HTTP/2

Definição de serviço
Arquivo .proto com geração automática de código

Tipos de comunicação
Unary, server streaming, client streaming e bidirectional

Desempenho
Alta performance, menor latência e menor uso de banda

Uso típico
Comunicação entre microsserviços e sistemas distribuídos

REST Arquitetura baseada em recursos usando HTTP/1.1 ou HTTP/2



Formato de dados
JSON (texto) ou outros (XML, etc.)

Protocolo
HTTP/1.1 ou HTTP/2

Definição de serviço
Uso de convenções de recursos (URLs e verbos HTTP)

Tipos de comunicação
Requisições e respostas (padrão request/response)

Desempenho
Maior overhead de dados e maior latência

Uso típico
APIs públicas, integração com navegadores e sistemas heterogêneos

UFC

Quando usar gRPC ou REST

gRPC

Mais adequado para comunicação entre serviços e cenários de alta performance.

Comunicação entre microsserviços
Em arquiteturas distribuídas.

Alta performance
Baixa latência e menor uso de banda.

Streaming de dados
Fluxos contínuos (unary, server, client e bidirectional).

Suporte a várias linguagens
Geração automática de código a partir do .proto.

APIs internas
Comunicação em ambientes controlados (rede privada).

REST

Mais adequado para APIs simples, públicas e com grande interoperabilidade.

APIs públicas
Exposição para clientes web, mobile e parceiros.

Simplicidade
Fácil de entender e testar (ex.: navegador, Postman).

Interoperabilidade
Suporte nativo em navegadores e ferramentas HTTP.

Baixo acoplamento
Uso de recursos padrão do HTTP (URLs, métodos, status).

Sistemas heterogêneos
Integração com clientes e plataformas diversas.

Demonstração

UFC

Calculadora

- Vamos preparar o ambiente. Então, execute os seguintes comandos (Linux/MacOS):
  - mkdir calc-grpc
  - cd calc-grpc
  - python3 -m venv venv
  - source venv/bin/activate
  - pip install grpcio grpcio-tools
  - code .

UFC

Calculadora

- Agora vamos criar o arquivo calculadora.proto:

```
syntax = "proto3";

package calculadora;

service Calculadora {
  rpc Somar (OperacaoRequest) returns (OperacaoResponse);
}

message OperacaoRequest {
  double numero1 = 1;
  double numero2 = 2;
}

message OperacaoResponse {
  double resultado = 1;
}
```

UFC

Calculadora

- Para gerar automaticamente os arquivos necessários para comunicação gRPC, execute o seguinte comando:
  - python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. calculadora.proto

UFC

Calculadora

- Agora vamos criar nosso arquivo servidor.py e cliente.py:
  - https://colab.research.google.com/drive/157lEhbZETa659olvAXPJxdx3WCJzYi35?usp=sharing

- Para executar:
  - python3 servidor.py
  - python3 cliente.py



UFC

engine lab

Dúvidas ??

E-mail: almada@crateus.ufc.br